



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Design

and

**Implementation of an Efficient Priority-Based Emergency Patient Management System
Using Appropriate Data Structures for Severity-Based Treatment and Arrival-Order
Processing**

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0307- Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

AIDS

Submitted by

K.Srinath(192524220)

D.Chaitanya Kumar(192411094)

Mohammed Fareeth S A(192511288)

Under the Supervision of

Dr.K.Kiruthika

Saveetha School of engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

COMMON COURSE ASSIGNMENT FORMAT

Data Structures – Hospital Patient Management System

Assignment Information

Particular	Details
Department	AI&DS
Programme	B.Tech
Course Code & Course Name	CSA0307 – Data Structures
Academic Year / Batch	
Faculty Name	Dr.K.Kiruthika
Assignment Title	Design and Implementation of a Data Structures-Based Hospital Patient Management System
Date of Issue	31-08-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) – CO	[Enter the applicable Course Outcome(s)]
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 3 – Good Health and Well-being; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Digital hospital information systems require reliable data organization, efficient searching, prioritization, record management and secure handling of information. The project demonstrates how data structures and algorithms can support these requirements.

B. Assignment Problem / Challenge

The challenge is to design a web-based Hospital Patient Management System that applies Data Structures and Algorithms to organize, process, search, prioritize and manage hospital-related records efficiently.

The system should demonstrate course concepts through practical modules such as patient management, registration, search and update, bed management, emergency handling, ICU/critical care, linked lists, appointments, medical records, billing, and payments.

The solution should use an interactive frontend for visualization and user interaction, a backend for business logic and data-structure operations, and a database for persistent storage. The design must consider correctness, usability, performance, security, reliability and maintainability.

C. Problem Statement

Problem / Case / Design Challenge:

Hospitals handle large amounts of patient and operational information. Managing patients, queues, appointments, beds, records and transactions using unstructured or inefficient methods can make searching, updating and prioritizing information difficult. The proposed Hospital Patient Management System addresses this problem by providing a centralized web application that uses appropriate Data Structures and Algorithms for efficient data processing.

The system should allow authorized users to manage patient records, register patients, search and update information, manage beds, handle emergency and critical-care cases, maintain appointments and medical records, and manage billing and payments. Data Structures such as queues, priority queues, linked lists, trees and graphs are incorporated where they are technically appropriate.

D. Requirements and Constraints

Functional requirements:

- Provide a dashboard for a quick overview of hospital activities and system status.
- Support patient registration, patient details, medical history and patient record management.
- Provide search and update functions for patient information.
- Manage general beds, ICU beds, occupied beds, bed assignment and bed transfer.
- Support emergency patient registration and emergency status tracking.
- Provide ICU/Critical Care views for ICU patients and critical cases.
- Demonstrate Linked List operations through an interactive module.
- Manage appointments and appointment history.
- Provide medical-record functions such as prescriptions, laboratory reports, treatment history, medical history, doctor notes and medication chart.
- Provide billing and invoice functions and payment/POS functions using Indian Rupees.
- Record important Data Structure operations for history and validation.

Performance requirements:

- Searching and updating should provide quick response for normal application-sized datasets.

- Queue and priority-queue operations should preserve the correct processing order.
- Traversal operations should produce correct and repeatable results.
- The interface should remain responsive while displaying Data Structure operations.

Technical constraints:

- Frontend: React with TypeScript and Vite.
- Backend: Java with Spring Boot, Spring Web and Spring Data JPA.
- Database: MySQL with JPA/Hibernate.
- Communication between frontend and backend through REST APIs.
- The Data Structures should be implemented in Java rather than being represented only by database tables.
- The application should be modular so that individual hospital and Data Structure modules can be maintained independently.

Security, privacy and reliability:

- Patient information should be treated as confidential and should not contain real personal information in demonstration data.
- Authentication and role-based access can be used to control access to protected functions.
- Passwords should not be stored as plain text; secure hashing such as BCrypt should be used in the backend.
- Invalid operations such as removing an item from an empty stack or queue should be handled with meaningful error messages.
- Database backups and appropriate validation should be considered for a production deployment.

E. Student Work

1. Problem Understanding and Formulation

What is the problem?

The problem is to create a structured and interactive system for hospital information management while demonstrating the practical use of Data Structures and Algorithms. The application must organize different types of records and provide efficient operations such as insertion, deletion, searching, traversal, prioritization and sorting.

What are the expected outcomes?

The expected outcome is a modular Hospital Patient Management System with an intuitive web interface, correctly implemented Data Structures, REST-based backend services and persistent database storage. The system should make the application of Data Structures visible through visualizations and operation results.

What information/data is available?

The system works with demonstration data such as patient identifiers, names, ages, priorities, statuses, appointment information, bed information, medical-record categories, billing information and Data Structure operation history. Sample data is used for testing instead of real patient information.

What assumptions are made?

- Users have authorized access to the system.
- Hospital staff enter valid patient and operational information.
- The MySQL database is available when persistent storage is required.
- The application is primarily an academic prototype and is not a replacement for a certified hospital information system.
- The number of records used for demonstration is manageable on a normal development computer.

What constraints must be satisfied?

The system must preserve data correctness, maintain appropriate processing order, provide a consistent user interface, protect sensitive information, handle invalid operations safely, and maintain separation between frontend, backend, Data Structure logic and database persistence.

2. Application of Course Knowledge

The project applies the following Data Structures and Algorithms concepts:

- Stack – used to demonstrate LIFO operations such as push, pop and peek.
- Queue – used to demonstrate FIFO processing and can support patient waiting-line management.
- Priority Queue – used to prioritize urgent and critical cases over normal cases.
- Linked List – used to demonstrate dynamic node-based insertion, deletion, searching and traversal.
- Binary Search Tree – used to demonstrate ordered storage, searching, insertion, deletion and tree traversals.
- Graph – used to represent relationships or connections and to demonstrate BFS and DFS traversal.
- Searching – Linear Search and Binary Search are used to compare different search approaches.
- Sorting – Bubble, Selection, Insertion, Merge and Quick Sort are used to demonstrate sorting algorithms.
- Time complexity – Big-O analysis is used to compare the expected efficiency of algorithms and operations.

The main reasoning is that different data organization problems require different structures. For example, a queue is appropriate for first-come-first-served processing, while a priority queue is appropriate when urgency determines processing order. A linked list demonstrates dynamic node manipulation, a binary search tree supports ordered searching, and a graph supports traversal of connected entities.

3. Solution / Design / Methodology

The proposed system follows a three-layer architecture:

- Frontend layer – React, TypeScript and Vite provide the user interface, forms, dashboards, navigation and Data Structure visualizations.
- Backend layer – Java Spring Boot provides REST APIs, business logic, validation and the actual Data Structure implementations.

- Database layer – MySQL provides persistent storage for users, patients, queue records and operation history.

The main Data Structure workflow is:

- The user selects a Data Structure and enters the required value or operation.
- The frontend sends the operation to the backend through a REST API.
- The Java backend performs the operation using the appropriate Data Structure.
- The backend returns the updated structure, operation result or error message.
- The frontend visualizes the result for the user.
- Important operations can be stored in the database for history and validation.

Example – Priority Queue for patient processing:

Patients are assigned a priority according to the academic demonstration rules. A critical case receives the highest priority, an urgent case receives a higher priority than a normal case, and normal cases receive lower priority. The backend priority queue determines the next case to be processed rather than relying only on frontend sorting.

Example – Stack visualization:

A stack can display elements from bottom to top and demonstrate push, pop and peek operations. The visualization changes after each successful backend operation, making the LIFO principle directly observable.

4. Use of Modern Tools

- React and TypeScript – development of reusable frontend components and type-safe user interfaces.
- Vite – frontend development and build tooling.
- Java – implementation of Data Structures, algorithms and backend business logic.
- Spring Boot – REST API development and backend application framework.
- Spring Data JPA / Hibernate – object-relational mapping and database access.
- MySQL – persistent storage for application records.
- Maven – dependency management and backend build automation.
- Git/GitHub – version control and project history, where available.
- Postman – testing REST API requests and responses.
- Browser developer tools – debugging frontend behavior and API communication.

Evidence of tool usage can include screenshots of the running application, Data Structure visualizations, REST API responses, database tables, test results and the project folder structure.

5. Results and Validation

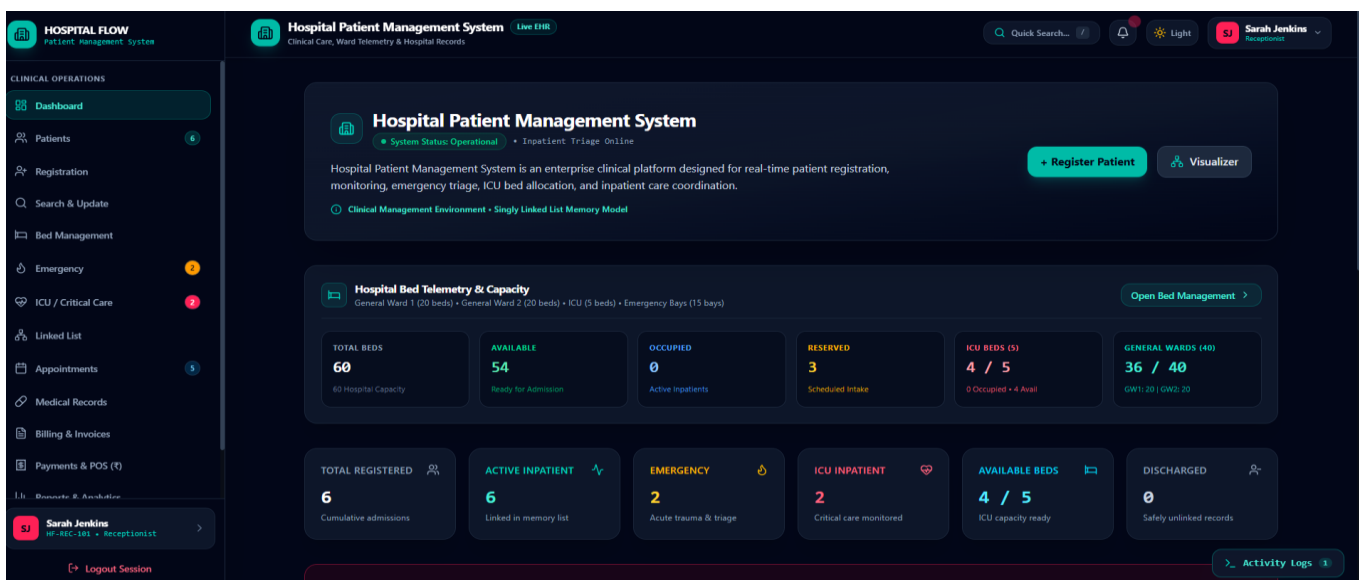
The proposed system is validated at three levels: user-interface validation, algorithm validation and data-persistence validation.

Functional test examples:

- Stack Push: insert an element and verify that it appears at the top.

- Stack Pop: remove the top element and verify LIFO behavior.
- Queue Enqueue: add an element and verify that it is placed at the rear.
- Queue Dequeue: verify FIFO behavior.
- Priority Queue: verify that higher-priority cases are processed first.
- Linked List: insert, delete, search and reverse nodes and verify the displayed sequence.
- BST: insert values and verify inorder traversal produces sorted output.
- Graph: create vertices/edges and verify BFS and DFS traversal orders.
- Searching: compare the result of linear and binary search for valid and invalid values.
- Sorting: verify that each sorting algorithm produces the correct ordered result.
- Patient module: create, view, update and delete demonstration patient records.
- Billing/payment modules: verify that records use Indian Rupees (₹) and are displayed consistently.

Validation should compare the application output with expected results for known test inputs. Error cases such as empty-stack pop, empty-queue dequeue, invalid search values and nonexistent records should also be tested.



Source code

Import

```
{
  ActiveTab, LinkedListState, OperationLogEntry, PatientData, PatientLocation, PatientPriority, PatientStatus
} from '../types';

import { TOTAL_ICU_BEDS, getOrderedNodes } from '../data/sampleData';
```

```

import { PatientManagementModal } from './PatientManagementModal';

interface PatientsViewProps {

  linkedList: LinkedListState;

  setLinkedList: React.Dispatch<React.SetStateAction<LinkedListState>>;

  addLog: (log: Omit<OperationLogEntry, 'id' | 'timestamp'>) => void;

  setDischargedCount: React.Dispatch<React.SetStateAction<number>>;

  showToast: (type: 'success' | 'error' | 'info' | 'warning', title: string, message: string) => void;

  setActiveTab: (tab: ActiveTab, subView?: string) => void;

}

export const PatientsView: React.FC<PatientsViewProps> = ({

  linkedList,

  setLinkedList,

  addLog,

  setDischargedCount,

  showToast,

  setActiveTab,

}) => {

  const [searchQuery, setSearchQuery] = useState("");

  const [locationFilter, setLocationFilter] = useState<string>('ALL');

  const [statusFilter, setStatusFilter] = useState<string>('ALL');

  const [priorityFilter, setPriorityFilter] = useState<string>('ALL');

  const [sortBy, setSortBy] = useState<'id' | 'name' | 'admissionDate' | 'priority' | 'room'>('id');

  const [sortOrder, setSortOrder] = useState<'asc' | 'desc'>('asc');

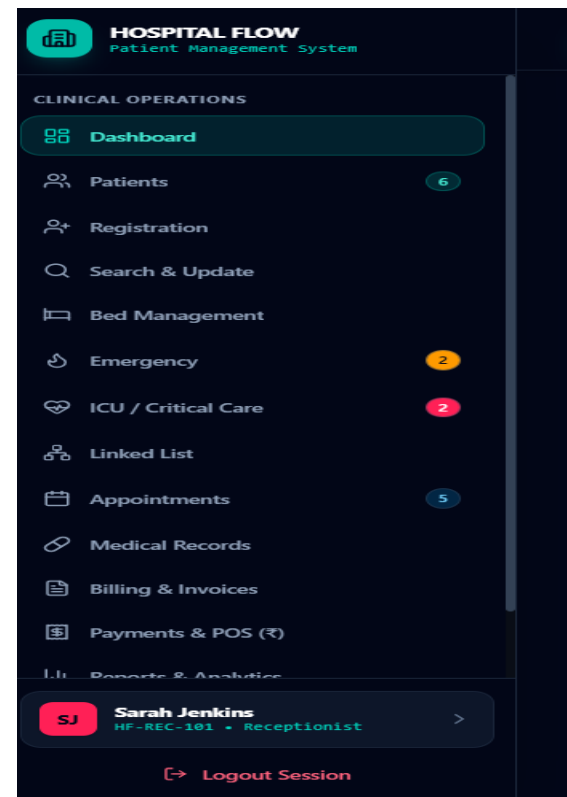
  const [selectedPatient, setSelectedPatient] = useState<PatientData | null>(null);

  const [isModalOpen, setIsModalOpen] = useState(false);

  const orderedNodes = getOrderedNodes(linkedList);

  const filteredPatients = useMemo(() => {

```




```

return orderedNodes

.map((n) => n.patient)

.filter((p) => {

const matchesSearch =

p.name.toLowerCase().includes(searchQuery.toLowerCase()) ||

p.id.toLowerCase().includes(searchQuery.toLowerCase()) ||

p.diagnosis.toLowerCase().includes(searchQuery.toLowerCase()) ||

p.doctor.toLowerCase().includes(searchQuery.toLowerCase()) ||

p.room.toLowerCase().includes(searchQuery.toLowerCase());

const matchesLocation = locationFilter === 'ALL' || p.location === locationFilter;

const matchesStatus = statusFilter === 'ALL' || p.status === statusFilter;

const matchesPriority = priorityFilter === 'ALL' || p.priority === priorityFilter;

return matchesSearch && matchesLocation && matchesStatus && matchesPriority;

})

.sort((a, b) => {

let compare = 0;

if (sortBy === 'id') {

compare = a.id.localeCompare(b.id, undefined, { numeric: true });

} else if (sortBy === 'name') {

compare = a.name.localeCompare(b.name);

} else if (sortBy === 'admissionDate') {

compare = a.admissionDate.localeCompare(b.admissionDate);

} else if (sortBy === 'room') {

compare = a.room.localeCompare(b.room);

} else if (sortBy === 'priority') {

const rank: Record<PatientPriority, number> = { Critical: 3, High: 2, Normal: 1 };

compare = (rank[a.priority] || 0) - (rank[b.priority] || 0);

```

```

}

return sortOrder === 'asc' ? compare : -compare;

});

}, [orderedNodes, searchQuery, locationFilter, statusFilter, priorityFilter, sortBy, sortOrder]);

const handleOpenManagement = (patient: PatientData) => {

setSelectedPatient(patient);

setIsModalOpen(true);

};

```

Patients Directory

View, search, triage, and manage active inpatient medical records

[+ Register Patient](#)

Location: All Units
Status: All Statuses
Priority: All Priorities

PATIENT RECORDS ROSTER
Showing 6 of 6 active records

PATIENT ID	PATIENT NAME	AGE	DIAGNOSIS	DOCTOR	ROOM	LOCATION	STATUS	PRIORITY	MANAGEMENT
101	Aarav Sharma Admitted: 2026-08-20	34 yrs	Acute Appendicitis (Post-Op)	Dr. Priya Nair	Bed G1-01	General Ward 1	Stable	Normal	Management
102	Rahul Verma Admitted: 2026-08-21	45 yrs	Post-Op Knee Arthroscopy	Dr. Marcus Vance	Bed G2-01	General Ward 2	Under Treatment	Normal	Management
103	Kiran Patel Admitted: 2026-08-23	29 yrs	Acute Respiratory Distress	Dr. Emily Chen	ER-01	Emergency Ward	Emergency	High	Management
104	Arjun Mehta Admitted: 2026-08-22	52 yrs	Severe Coronary Syndrome	Dr. Sarah Patel	ICU-01	ICU	Critical	Critical	Management
105	Vikram Rao Admitted: 2026-08-23	61 yrs	Septic Shock & Renal Care	Dr. Robert Davis	ICU-02	ICU	Critical	Critical	Management
106	Ananya Gupta Admitted: 2026-08-23	38 yrs	Blunt Abdominal Trauma	Dr. Alan Sterling	ER-02	Emergency Ward	Emergency	High	Management

[> Activity Logs](#)

```

const getStatusBadge = (status: PatientStatus) => {

switch (status) {

case 'Stable':

return 'bg-emerald-500/15 text-emerald-300 border-emerald-500/30';

case 'Under Treatment':

return 'bg-teal-500/15 text-teal-300 border-teal-500/30';

case 'Emergency':

```

```
return 'bg-amber-500/15 text-amber-300 border-amber-500/30';

case 'Critical':

return 'bg-rose-500/15 text-rose-300 border-rose-500/30 animate-pulse';

case 'Discharged':

return 'bg-slate-500/15 text-slate-300 border-slate-500/30';

default:

return 'bg-slate-800 text-slate-300 border-slate-700';

}

};
```

```
const getPriorityBadge = (priority: PatientPriority) => {

switch (priority) {

case 'Normal':

return 'bg-slate-800 text-slate-300 border-slate-700';

case 'High':

return 'bg-amber-500/15 text-amber-300 border-amber-500/30 font-semibold';

case 'Critical':

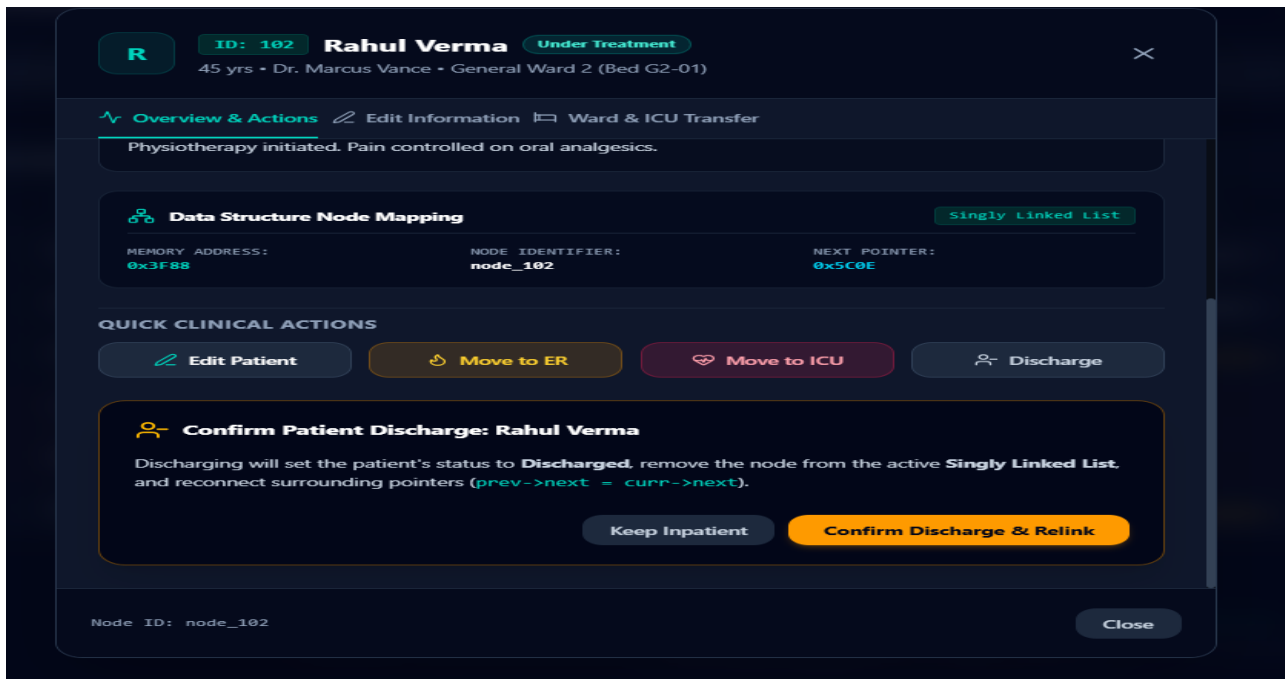
return 'bg-rose-500/20 text-rose-300 border-rose-500/40 font-bold';

default:

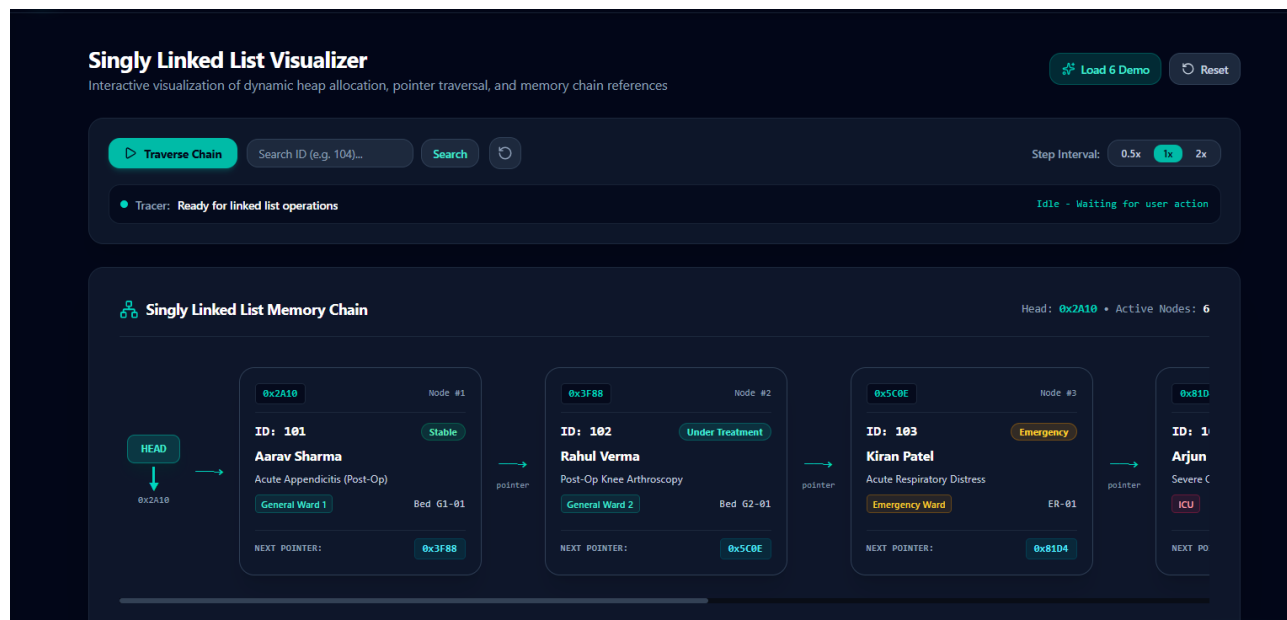
return 'bg-slate-800 text-slate-300';

}

};
```



Visulaizer



6. Analysis and Engineering Decision

Different Data Structures were selected according to the type of operation they support. A single structure is not optimal for every hospital-management task. The design therefore uses multiple structures to demonstrate appropriate course concepts.

Advantages:

- Provides a practical demonstration of Data Structures rather than only theoretical examples.

- Separates presentation, business logic and persistent storage.
- Makes algorithm behavior visible through interactive visualizations.
- Supports modular development and future extension.
- Provides an operation-history mechanism for academic demonstration and testing.

Limitations:

- The prototype is intended for academic use and does not provide the full compliance, auditing and infrastructure requirements of a real hospital information system.
- Performance results depend on dataset size and development-machine resources.
- The visualization layer is designed for learning and may simplify some real-world workflows.

The selected architecture is justified because it clearly separates responsibilities: React handles presentation, Spring Boot handles application logic and Data Structures, and MySQL handles persistent records. This separation also makes the system easier to test and maintain.

7. Broader Considerations

Sustainability: A digital record system can reduce unnecessary paper-based record handling and support more organized information management.

Environment: Digital workflows can reduce physical paperwork and storage requirements, although the computing infrastructure itself consumes energy.

Society: Efficient patient information management can support organized service delivery and reduce avoidable delays in information retrieval.

Safety: Validation, access control and correct priority handling are important because incorrect patient or priority information can lead to inappropriate processing.

Ethics: Patient information must be handled confidentially. Demonstration data should be fictional, and access should be limited according to user roles.

Economics: A modular software design can reduce future maintenance effort and allow additional modules to be introduced without redesigning the complete system.

Accessibility: The interface should use readable text, clear labels, adequate contrast and consistent navigation so that users can operate the system effectively.

Professional responsibility: Developers must test algorithms, document limitations, protect data, avoid misleading claims about clinical capability and follow appropriate software engineering practices.

8. Conclusion

Proposed solution: A modular web-based Hospital Patient Management System that combines a React/TypeScript frontend, Java Spring Boot backend, MySQL database and practical Data Structure implementations.

Major findings: Stack, Queue, Priority Queue, Linked List, Binary Search Tree, Graph, searching and sorting algorithms can be incorporated into meaningful hospital-management scenarios. The project demonstrates how choosing an appropriate Data Structure affects organization, processing and retrieval of information.

Achievement of requirements: The proposed architecture addresses patient management, queue and priority processing, records, appointments, beds, emergency/critical care, billing and payment-related workflows while keeping Data Structure concepts central to the implementation.

Limitations: The project is an academic prototype and does not claim to provide a production-ready hospital information system or clinical decision support.

Possible improvements: Future work can include stronger authentication, role-based permissions, audit logs, automated backups, cloud deployment, larger-scale performance testing, accessibility improvements and more advanced visualization of algorithm complexity.

9. Student Reflection

This assignment provides practical understanding of how Data Structures can be connected to a real-world software problem. Instead of treating Stack, Queue, Linked List, Trees and Graphs as isolated classroom topics, the project shows how their properties influence application design. It also provides experience in separating frontend presentation from backend processing and persistent database storage.

With additional time and resources, the project could be improved by adding comprehensive automated testing, stronger authentication and authorization, detailed performance benchmarking for different algorithms, improved accessibility, more advanced graph/tree visualization, database backup mechanisms and deployment to a controlled cloud environment.

10. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. – Introduction to Algorithms.
- Horstmann, C. S. – Core Java / Java programming references.
- Spring Boot and Spring Data JPA official documentation.
- MySQL official documentation.
- React and TypeScript official documentation.
- Vite official documentation.
- Project source code, screenshots, test outputs and AI-assisted development tools used during implementation should be acknowledged where applicable.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

The exact CO and PO mapping should be completed according to the official course outcome and programme-outcome definitions supplied by the faculty. The template's assessment distribution is retained below.

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	[Enter CO]	[Enter PO(s)]	L3/L4	10
Application of knowledge	[Enter CO]	[Enter PO(s)]	L3/L4	20
Solution / Design / Implementation	[Enter CO]	[Enter PO(s)]	L4/L5/L6	20
Modern tool usage	[Enter CO]	[Enter PO(s)]	L3/L4	10
Validation	[Enter CO]	[Enter PO(s)]	L4/L5	15
Analysis & justification	[Enter CO]	[Enter PO(s)]	L4/L5	15
Broader considerations	[Enter CO]	[Enter PO(s)]	L4/L5	5
Documentation & reflection	[Enter CO]	[Enter PO(s)]	L3/L4	5

H. Faculty Evaluation & Continuous Improvement CO Attainment

Target CO Attainment: [To be completed by faculty]

Actual CO Attainment: [To be completed after evaluation]

Faculty Analysis: [To be completed by faculty based on student performance, strengths, weaknesses and assessment evidence.]

Corrective / Improvement Action: [To be completed by faculty.]

Action to be implemented in the next offering: [To be completed by faculty.]

Documents to be Maintained

The following evidence should be maintained for the assignment:

- Assignment question/problem statement.
- CO–PO and Bloom's mapping.
- Assessment rubric.
- Student submission/report.
- Screenshots of the working Hospital Patient Management System.
- Evidence of Data Structure operations and algorithm outputs.
- REST API or backend testing evidence, where applicable.
- Database schema and sample database evidence, where applicable.
- Evaluated scripts/reports.
- Marks and rubric-wise assessment.
- CO attainment analysis.
- Sample student work representing different performance levels.
- Faculty analysis and continuous-improvement action.

Detailed Project Description

The Hospital Patient Management System is a web-based academic project developed to demonstrate the practical application of Data Structures and Algorithms in a real-world inspired environment. The system combines an interactive user interface with structured data-processing operations so that students can understand how theoretical concepts such as stacks, queues, priority queues, linked lists, trees, graphs, searching and sorting can be applied to practical information-management tasks.

The application is designed as a modular system rather than as a collection of unrelated programs. Different hospital activities are represented as separate functional areas, while the Data Structures provide the computational foundation for organizing and processing information. This makes the project suitable for demonstrating both software development and core Data Structures concepts during laboratory evaluation, project demonstration and viva examination.

1. Project Objectives

- To develop an interactive Hospital Patient Management System that demonstrates the practical use of Data Structures.
- To provide a centralized interface for managing patient-related and hospital operational information.
- To demonstrate FIFO processing using Queue and urgency-based processing using Priority Queue.
- To demonstrate LIFO behavior using Stack operations such as push, pop and peek.
- To demonstrate dynamic data organization using Linked List operations.
- To demonstrate ordered storage and traversal using a Binary Search Tree.
- To demonstrate graph representation and BFS/DFS traversal.
- To compare searching and sorting algorithms through actual execution and output.
- To separate frontend presentation, backend processing and database persistence.
- To provide a foundation that can be extended with additional modules in future versions.

2. Scope of the Application

The scope of the proposed application covers the major information-management activities represented in the current project interface. The system contains modules for Dashboard, Patients, Registration, Search and Update, Bed Management, Emergency, ICU/Critical Care, Linked List, Appointments, Medical Records, Billing and Invoices, and Payments & POS. Each module is intended to provide a focused area of functionality while maintaining a common application structure.

The project is primarily an academic prototype. It demonstrates software architecture, user-interface design, Data Structures, algorithms, database concepts and backend development. It should not be considered a certified clinical or production hospital information system. Real patient information should not be used for demonstrations or testing.

3. Existing Application Modules

Dashboard: Provides an overall entry point and summary-oriented view of the application. It can present important counts, system status and shortcuts to frequently used modules.

Patients: Provides access to patient-related information such as the patient list, patient details and medical history. These functions remain grouped within the existing Patients section.

Registration: Supports new patient registration, updating patient information and viewing patient records.

Search & Update: Provides mechanisms for locating patient information and updating relevant details.

Bed Management: Supports bed availability, occupied beds, assignment and transfer operations. The interface also represents ICU capacity separately.

Emergency: Supports emergency patient registration, emergency patient lists and emergency-status information.

ICU / Critical Care: Provides access to ICU patients, critical cases and ICU bed availability.

Linked List: Provides an academic Data Structure demonstration using linked nodes and operations such as insertion, deletion, searching and traversal.

Appointments: Supports upcoming appointment information and appointment history.

Medical Records: Provides access to prescriptions, laboratory reports, treatment history, medical history, doctor notes and medication-chart information.

Billing & Invoices: Provides patient bills, hospital bill generation, pending dues and billing history.

Payments & POS (₹): Provides POS billing, bill collection, payment records and refund management. Monetary values are represented using Indian Rupees.

4. Data Structures Used in the Project

4.1 Stack

A Stack follows the Last In, First Out principle. The element inserted most recently is the first element removed. In the project, Stack is useful as an educational module for demonstrating push, pop and peek operations. The interface can visualize the stack vertically so that the top element is clearly identified.

The main operations are push, pop, peek, isEmpty, size and clear. A push operation adds an element to the top. A pop operation removes the top element. Peek returns the top element without removing it. The expected time complexity for push and pop in an array-based stack is $O(1)$, making Stack suitable for fast insertion and removal at one end.

4.2 Queue

A Queue follows the First In, First Out principle. The element that enters first is normally processed first. This makes a Queue appropriate for demonstrating waiting-line management. In the hospital scenario, a basic queue can represent patients waiting for a service when no additional priority rule is applied.

The main operations are enqueue, dequeue, front, rear, isEmpty and size. Enqueue inserts an item at the rear, while dequeue removes an item from the front. Queue operations can normally be implemented in $O(1)$ time when an appropriate front/rear representation is used.

4.3 Priority Queue

A Priority Queue extends the basic queue concept by considering priority when selecting the next item. This is particularly relevant to emergency and critical-care demonstrations. A critical case can be assigned a higher priority than an urgent or normal case. The backend should perform the priority operation so that the demonstration represents an actual algorithmic implementation rather than simply changing the display order in the frontend.

Priority queues can be implemented using structures such as heaps. For an academic prototype, the implementation can focus on correct insertion, priority comparison and removal of the highest-priority item. The project should clearly document the priority rules used for demonstration.

4.4 Linked List

A Linked List consists of nodes in which each node stores data and a link to another node. It is useful for demonstrating dynamic memory organization and pointer/reference-based relationships. The project can provide insertion at the beginning, insertion at the end, insertion at a selected position, deletion, searching, display and reversal.

Unlike an array, a linked list does not require elements to occupy contiguous memory locations. Insertion or deletion at a known node position can be efficient because elements do not have to be shifted. However, searching generally requires sequential traversal and therefore has $O(n)$ time complexity.

4.5 Binary Search Tree

A Binary Search Tree organizes values according to an ordering rule: values smaller than a node are stored in its left subtree and larger values are stored in its right subtree, subject to the implementation's duplicate-handling rule. The project can demonstrate insertion, search, deletion and tree traversals.

The three important depth-first traversals are inorder, preorder and postorder. Inorder traversal of a valid BST produces values in sorted order. Search, insertion and deletion have average-case behavior close to $O(\log n)$ in a reasonably balanced tree, but can degrade to $O(n)$ in a highly unbalanced tree.

4.6 Graph

A Graph represents entities as vertices and relationships or connections as edges. The project can use a graph module to demonstrate how connected information can be represented and traversed. Breadth First Search explores neighboring vertices level by level, while Depth First Search explores one path more deeply before backtracking.

BFS is commonly implemented with a Queue, while DFS can be implemented using recursion or a Stack. For a graph represented using adjacency lists, BFS and DFS have typical time complexity of $O(V + E)$, where V is the number of vertices and E is the number of edges.

5. Searching Algorithms

Searching is the process of locating a required element within a collection. The project can demonstrate both Linear Search and Binary Search and compare their requirements and performance.

Linear Search checks elements one by one until the required value is found or the collection ends. It does not require sorted input and has $O(n)$ worst-case time complexity. Binary Search repeatedly divides a sorted search range into two parts and therefore has $O(\log n)$ time complexity, but it requires the data to be appropriately sorted and indexed for the chosen implementation.

6. Sorting Algorithms

Bubble Sort: Repeatedly compares adjacent elements and swaps them when they are in the wrong order. It is simple to understand but generally inefficient for large datasets, with $O(n^2)$ average and worst-case complexity.

Selection Sort: Repeatedly selects the smallest remaining element and places it in its correct position. Its average and worst-case time complexity is $O(n^2)$.

Insertion Sort: Builds a sorted portion of the collection by inserting each new element into its appropriate position. It can perform well for small or nearly sorted datasets, with $O(n^2)$ worst-case complexity.

Merge Sort: Divides the collection into smaller parts, sorts the parts and merges them. Its time complexity is $O(n \log n)$ and it provides predictable performance.

Quick Sort: Selects a pivot and partitions the collection around it. Its average time complexity is $O(n \log n)$, while poor pivot choices can lead to $O(n^2)$ worst-case behavior.

7. Frontend Design

The frontend is implemented using React with TypeScript and Vite. React supports component-based development, allowing the dashboard, navigation, forms, cards, tables and Data Structure visualizations to be organized into reusable components. TypeScript provides type checking and makes the application easier to maintain as the number of modules grows.

The sidebar provides access to the major functional areas. The internal options of each major section can remain within the relevant page instead of making the global sidebar excessively long. This creates a cleaner navigation structure while preserving the existing functionality of the application.

Interactive visualizations are especially important for a Data Structures course. When a user performs an operation such as push, pop, enqueue, dequeue, insertion or traversal, the interface should clearly show the resulting state. This helps connect the algorithmic operation with the visual representation.

8. Backend Design

The recommended backend is Java with Spring Boot. Spring Boot can expose REST endpoints that receive requests from the React frontend, execute application logic and return structured JSON responses. The backend should contain separate controller, service, model, repository and Data Structure packages.

Controllers handle HTTP requests, services contain application logic, model classes represent persistent entities, repositories communicate with MySQL through JPA, and Data Structure classes contain the actual algorithm implementations. This separation follows a layered design and reduces unnecessary coupling between the user interface and the underlying algorithms.

For example, when the frontend requests a stack push operation, the request is received by a StackController. The controller delegates the operation to StackService, which uses the Java Stack implementation. The updated state is then returned to the frontend. If operation history is enabled, the relevant event can also be stored in MySQL.

9. Database Design

MySQL is recommended as the relational database for persistent information. The database should store information that must remain available after the application is restarted. Examples include user accounts, patient demonstration records, queue records and operation history.

- users – stores user identifiers, names, email addresses, roles and securely handled authentication information.
- patients – stores demonstration patient information such as identifier, name, age, priority and status.
- queue_records – records queue-related information and its relationship to patients.
- operations – stores Data Structure operation history, including operation type, Data Structure, input, result and timestamp.

The database should not be treated as a replacement for the Data Structures. The Java backend should execute the algorithms, while MySQL should provide persistence where required. This distinction is important for a Data Structures course because it demonstrates the actual implementation and behavior of the structures.

10. Frontend–Backend–Database Communication

The proposed communication flow is React frontend → REST API → Spring Boot backend → Data Structure/business logic → JPA/Hibernate → MySQL. JSON can be used as the common data format for requests and responses.

This architecture provides a clear separation of responsibilities. The frontend is responsible for presentation and interaction, the backend is responsible for processing and validation, and the database is responsible for persistence. Such separation makes the system easier to debug, test, extend and maintain.

11. Example Operational Flow – Patient Queue

- A patient is entered into the application with a relevant priority or status.
- The frontend sends the information to the backend.
- The backend validates the input and inserts the patient into the appropriate Queue or Priority Queue.
- The Data Structure determines the correct processing order.
- The backend returns the updated queue state.
- The frontend displays the queue visually.
- The operation can be recorded in the operation-history table for demonstration and audit purposes.

12. Example Operational Flow – Linked List

- The user enters a value and selects an insertion operation.
- The frontend sends the operation to the backend.
- The Java Linked List creates a node and connects it at the required position.
- The backend returns the updated node sequence.
- The frontend renders the linked nodes and their connections.
- Deletion and search follow the same request-processing-response pattern.

13. User Roles and Security

A basic role-based approach can distinguish administrators, staff users and student/demo users. The exact permissions should depend on the academic requirements. Authentication should be implemented securely, and passwords should be hashed rather than stored as plain text.

The project should use fictional demonstration data. Real patient identifiers, medical details, addresses or other personally identifiable information should not be included in screenshots, sample databases or public repositories. Access to sensitive operations should be restricted where appropriate.

14. Testing Strategy

Testing should cover both individual Data Structure operations and complete user workflows. Unit-style tests can validate the Java algorithms, while integration tests can validate REST endpoints and database interaction. Frontend testing can verify that buttons, navigation and visualizations respond correctly.

- Positive testing – verify correct behavior for valid inputs.
- Negative testing – verify meaningful errors for invalid inputs.
- Boundary testing – test empty structures, single-element structures and larger demonstration inputs.

- Integration testing – verify frontend requests reach the correct backend endpoint and return expected results.
- Database testing – verify that records are inserted, retrieved, updated and deleted as intended.
- Usability testing – verify that navigation labels, buttons, forms and messages are understandable.
- Regression testing – verify that new backend or navigation changes do not break existing application functionality.

15. Sample Test Cases

Test Case	Input	Expected Result	Status
Stack Push	10	10 becomes the top element	To be tested
Stack Pop	[10,20], pop	20 is removed first	To be tested
Queue Enqueue	Patient A	Patient A is added at rear	To be tested
Queue Dequeue	[A,B], dequeue	A is removed first	To be tested
Priority Queue	Normal + Critical	Critical is selected first	To be tested
Linked List Search	Search 30	Correct node/index is reported	To be tested
BST Inorder	40,20,60	20,40,60	To be tested
Graph BFS	Connected sample graph	Correct level-order traversal	To be tested
Binary Search	Sorted array, target 50	Target found at correct position	To be tested
Empty Stack Pop	Empty stack	Meaningful error message	To be tested

16. Complexity Analysis

Operation / Algorithm	Best Case	Average Case	Worst Case
Stack Push/Pop	$O(1)$	$O(1)$	$O(1)$
Queue Enqueue/Dequeue	$O(1)$	$O(1)$	$O(1)$
Linked List Search	$O(1)$	$O(n)$	$O(n)$
BST Search	$O(1)$	$O(\log n)$	$O(n)$
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$
Bubble Sort	$O(n)^*$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
BFS / DFS	$O(V+E)$	$O(V+E)$	$O(V+E)$

* Bubble Sort can achieve $O(n)$ in an optimized implementation when the input is already sorted.

17. Advantages of the Proposed System

- Provides a practical environment for learning Data Structures.
- Combines theoretical algorithms with visible, interactive results.
- Uses modular frontend design that can be extended with additional pages.
- Separates backend logic from frontend presentation.
- Provides persistent database storage for selected records and history.

- Supports multiple hospital-management scenarios within one application.
- Makes algorithm correctness easier to demonstrate during a viva.
- Can be extended to include additional Data Structures and algorithms.

18. Limitations

- The project is an academic prototype and should not be used for real clinical decisions.
- Real hospital deployment would require extensive security, privacy, compliance and reliability controls.
- Algorithm performance may differ with very large datasets and different implementation choices.
- A demonstration database does not represent the complexity of a production hospital database.
- Internet or server configuration may be required when frontend and backend are deployed separately.

19. Future Enhancements

- Add comprehensive role-based authorization for different hospital staff.
- Add audit logging for important record changes.
- Add automated database backup and recovery mechanisms.
- Add advanced Data Structure visualization with step-by-step animation.
- Add performance charts comparing sorting and searching algorithms.
- Add pagination and filtering for larger patient datasets.
- Add automated unit and integration test suites.
- Add cloud deployment with appropriate security controls.
- Add accessibility improvements and keyboard-friendly navigation.
- Add configurable hospital workflow rules without changing core Data Structure implementations.

20. Expected Learning Outcomes

- Understand how abstract Data Structures can be applied to practical software problems.
- Implement fundamental Data Structures and algorithms in Java.
- Analyze time complexity and select appropriate structures for different tasks.
- Understand the role of REST APIs in connecting frontend and backend applications.
- Understand relational database persistence using MySQL and JPA/Hibernate.
- Develop modular software using separation of concerns.
- Perform functional and algorithmic testing.
- Recognize the importance of security, privacy, validation and professional responsibility in software systems.

21. Overall Conclusion

The Hospital Patient Management System provides a suitable real-world context for demonstrating Data Structures and Algorithms. The project moves beyond isolated console programs by placing Stack, Queue, Priority Queue, Linked List, Binary Search Tree, Graph, searching and sorting operations within an interactive web application.

The recommended architecture uses React with TypeScript and Vite for the frontend, Java Spring Boot for the backend, and MySQL for persistent database storage. This separation makes the system easier to understand and provides a clear explanation of how user actions are translated into backend operations and stored data.

The project also emphasizes that selecting a Data Structure should depend on the problem being solved. Queues are appropriate for FIFO processing, Priority Queues for urgency-based processing, Stacks for LIFO behavior,

Linked Lists for node-based dynamic organization, BSTs for ordered searching and Graphs for connected relationships. This makes the application directly relevant to the learning objectives of a Data Structures course.

A. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

B.CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1	PO1, PO2	L3/L4	10
Application of knowledge	CO1, CO2	PO1, PO2	L3/L4	20
Solution / Design / Implementation	CO1, CO2	PO1, PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage	CO1	PO5	L3/L4	10
Validation	CO1, CO2	PO2, PO4	L4/L5	15
Analysis & justification	CO2	PO2, PO4	L4/L5	15
Broader considerations	CO2	PO6, PO8	L4/L5	5
Documentation & reflection	CO1, CO2	PO10, PO12	L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by this particular assignment. Every assignment is not required to map to every PO.

C.Faculty Evaluation & Continuous Improvement

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering:

Documents to be Maintained

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable

6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action

